


Class Syntax me inheritance hoti hai.

What is inheritance?

```
class Person{
  constructor(naam , umar){
    this.username = naam;
    this.age = umar;
  }
  printName(){
    console.log(`my name is ${this.username}`)
  }
  getName(){
    return `naam is ${this.username}`
  }
}

class Student{
  constructor(naam , umar){
    this.username = naam;
    this.age = umar;
  }
  printName(){
    console.log(`my name is ${this.username}`)
  }
  getName(){
    return `naam is ${this.username}`
  }
}

let student1 = new Student('samarth' , 120);
console.log(student1);
console.log(student1.printName());
console.log(student1.getName());
```

Hamare code me 2 same classes hai but with different names. This is making code lengthy. How to reduce the length of this code? you can use this code too but it is not a good way of

(20:00)

writing code.

Ab class Student ko khaali kar do. Ab hum class Person ki sabhi properties ko class Student mei extract kar lenge.

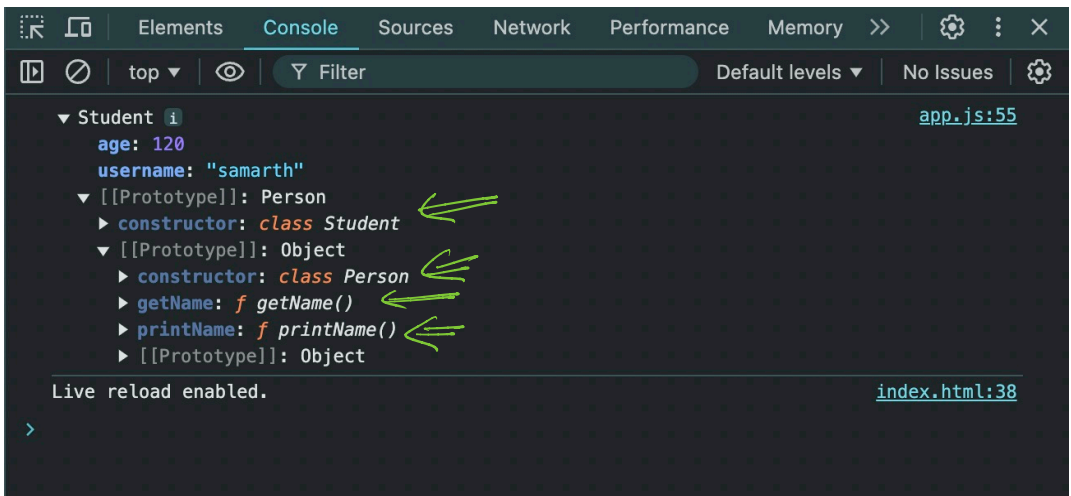
class Student extends Person {
}
↓
matlab?

class Student is extracting properties of class Person

or inherit kar rahi hai
↓

done same hi hai (bas words different hai)

```
class Person{  
  constructor(naam , umar){  
    this.username = naam;  
    this.age = umar;  
  }  
  printName(){  
    console.log(`my name is ${this.username}`)  
  }  
  getName(){  
    return `naam is ${this.username}`  
  }  
}  
  
class Student extends Person{  
}  
  
let student1 = new Student('samarth' , 120);  
console.log(student1);
```



"extends" inherit kar lega properties of class Person to class Student.

Agar maan lo mujhe extra property chahiye jaise rollNo tab kya karenge?

we will use Super ()

```
class Student extends Person{  
  constructor(naam, umar, rollNo){  
    super(naam, umar); //parent  
    this.rollnumber = rollNo;  
  }  
}
```

To chije parent mei se same use kar sake ho, write them inside super (naam, umar)

To jo chije mujhe Person mei se chahiye, mai use super ke brackets mei likh dunga.


```
class Person{
  constructor(naam , umar){
    this.username = naam;
    this.age = umar;
  }
  printName(){
    console.log(`my name is ${this.username}`)
  }
  getName(){
    return `naam is ${this.username}`
  }
}

class Student extends Person{
  constructor(naam , umar, rollNo){
    super(naam, umar);
    this.rollNumber = rollNo;
  }
}

let student1 = new Student('samarth' , 120, 1008);
console.log(student1);

console.log(student1.printName()); → ll
```

```
► Student {username: 'samarth', age: 120, rollNumber: 1008} app.js:83
my name is samarth app.js:68
undefined app.js:85
>
```

hamne ll par dekha ki printName() bhi
inherit hogya hai.

```

class Person{
  constructor(naam , umar){
    this.username = naam;
    this.age = umar;
  }
  printName(){
    console.log(`my name is ${this.username}`)
  }
  getName(){
    return `naam is ${this.username}`
  }
}

```

```

class Student extends Person{
  constructor(naam , umar, rollNo){
    super(naam, umar);
    this.rollNumber = rollNo;
  }

```

```

  printName(){
    console.log(`naamm is ${this.username}`)
  }

```

```

  alag(){
    console.log('haa mai alag');
  }

```

→ added a new function

```

let student1 = new Student('samarth' , 120, 1008);
console.log(student1);

```

```

console.log(student1.printName()); → l1

```

```

console.log(student1.alag()); → l2

```

```

> Student {username: 'samarth', age: 120, rollNumber: 1008} app.js:119
naamm is samarth app.js:110
undefined app.js:121
haa mai alag app.js:114
undefined app.js:123

```

l1 → override kar diya Person ke function printName() ko.

we added a new function alag() and it worked.

Agar mujhe class Student ^{mei} ~~mei~~ ^{mei} function add karna hai, naaki rollNo, then mai ~~self~~ ^{mei} function add karunga.

```
class Student extends Person{  
  printName(){  
    console.log('naammm is ${this.username}')  
  }  
  
  alag(){  
    console.log('haa mai alag');  
  }  
}
```

Arrow Function :-

↳ shorthand syntax for writing functions.

```
function sum(a,b){  
  return a+b;  
}  
  
console.log(sum(5,6))
```

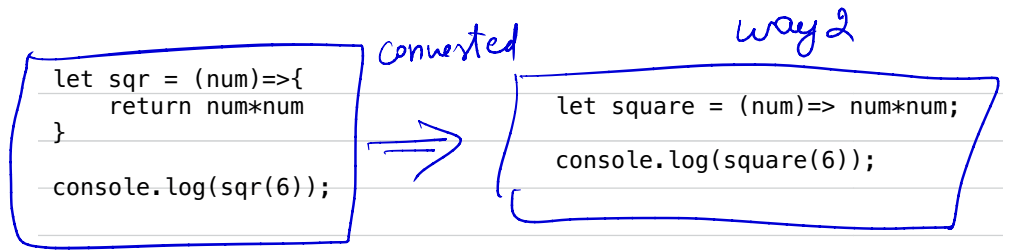


way 1

```
let sum = (a,b) => {  
  return a+b;  
}  
  
console.log(sum(5,6))
```

the function.

- arrow function ko variable mei store karro to call.
- there is no need to give name to function and also you don't write "function" keyword.
- use arrow after bracket.



jab apke code mei sirf ek line hai function ke andar return keliye; you can avoid writing braces & return.

o/p dono mei same hoga.

way 3 :-

```
let sqr = num => num * num  
  
console.log(sqr(6));
```

yaha hamne parenthesis () hata diye; jab hamare paas sirf ek argument ho, tab hum aisa krte hai.

Break over :- (1:17:00)

PHASE 3 of JS Begins Here :-

Asynchronous Programming :-

JS → ① interpreted

② synchronous (ek ke baad ek)

③ loosely typed

④ dynamically typed.

⑤ Single Threaded

single call stack hai isliye single threaded hai JS.

"Please Refer Shreyansh Sir Lec 16
Asynchronous Programming Topic Notes"
maine sirf important topics ke notes
banaye hai.

JS engine kisi ka integer nahi karta, jaise hi ise
code milega use line by line execute
kar dega. Maam to mai JS ko wait karana
chahata hu, wo kaise hoga?

setTimeout() use karenge hum to create
delay:-

Syntax :- setTimeout (cb fnc , delay);

↳ %n milli
second

1 sec = 1000 ms

```
console.log('start')  
setTimeout(function(){  
  console.log("run after 4 seconds")  
}, 4000)  
console.log('end')
```

→ this will run after
4 seconds.

l1→
l2→
l3→

But yaha ek ajeab chiz hui, pehle l1 print hui, then l2, then after 4sec l3 print hua.

Balki console (end) to sabse last mei likha hai. Mainne to pada tha ki line by line execution hota hai. Ei ka bawal hai? Also, sir ne ye bhi bola tha ki JS never waits but here it is. `setTimeout()` is not a method → it is a part of your browser. `setTimeout()` is a web API of browser.

`setTimeout()` → is not a part of JavaScript.

↳ we are able to use it kyunki hum browser ka console use kar sakte hai.

(1:39:00)

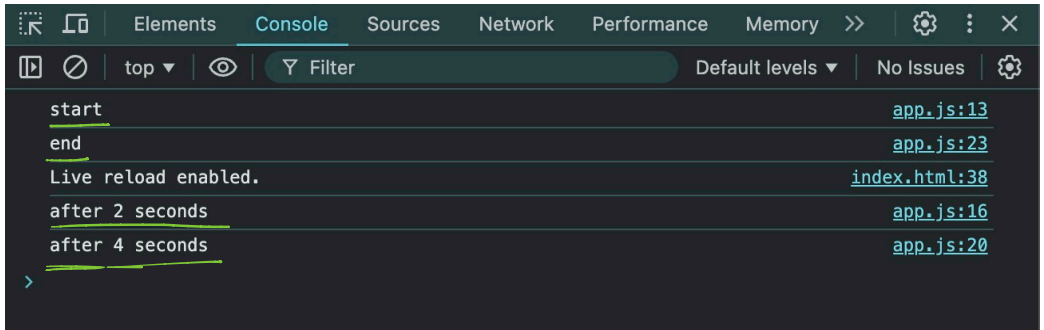
I prefer explanation of Shreyance on "webapi, eventloop, queue" topic. ~~Explan-~~ Explanations of Samarth, Shreyance, Gpt → All three are different.

Q guess the o/p :-

also ye bhi batavo ki iss program
ko execute hone mei
kitna time laga hoga?

```
console.log('start');  
  
setTimeout(function(){  
    console.log('after 2 seconds')  
} , 2000);  
  
setTimeout(function(){  
    console.log('after 4 seconds')  
} , 4000);  
  
console.log('end');
```

Ans 4 seconds



```
start app.js:13  
end app.js:23  
Live reload enabled. index.html:38  
after 2 seconds app.js:16  
after 4 seconds app.js:20  
>
```

Q

```
console.log('start');  
  
setTimeout(function(){  
    console.log('after 4 seconds')  
} , 4000);  
  
setTimeout(function(){  
    console.log('after 2 seconds')  
} , 2000);  
  
console.log('end');
```

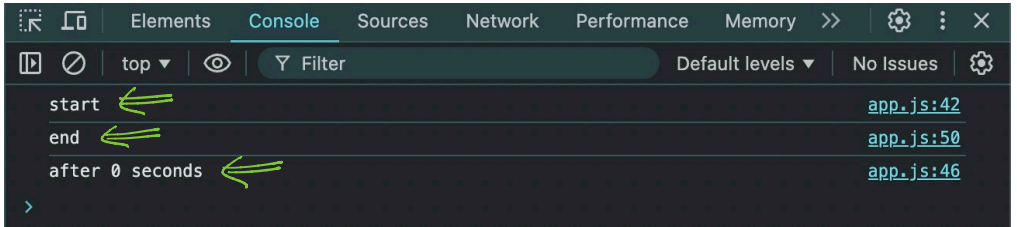
→ same o/p like
last question

Q

```
console.log('start')
```

```
setTimeout(function(){  
    console.log('after 0 seconds')  
}, 0)
```

```
console.log('end')
```



This is asynchronous behavior. Kunki JS kisi ka wait nahi karti, Aap setTimeout() se timer laga do, fir bhi isi beech next line execute ho jayegi. PLS check ss given below for more info.

my teacher told me JS waits for nothing. did you understand that

Yes, your teacher is referring to JavaScript's non-blocking, asynchronous nature, particularly in the context of the browser or Node.js runtime environment.

When they say "JS waits for nothing," they mean:

1. **Single-threaded model:** JavaScript runs on a single thread. If a task takes too long (like fetching data from an API or reading a file), it doesn't block the execution of the rest of the code. Instead, it delegates such tasks to the browser or Node.js event loop.
2. **Asynchronous handling:** JavaScript uses mechanisms like callbacks, promises, and `async/await` to handle tasks asynchronously. The rest of the code keeps executing while waiting for those tasks to complete in the background.

Here's an example to demonstrate:

javascript

Copy code

```
console.log("Start");  
  
setTimeout(() => {  
    console.log("This is asynchronous");  
}, 2000);  
  
console.log("End");
```


Output:

sql

 Copy code

Start

End

This is asynchronous

Explanation:

1. The `console.log("Start")` runs first.
2. The `setTimeout` function sets up a timer but doesn't block execution. The callback is queued to execute after 2 seconds.
3. `console.log("End")` runs next because JavaScript doesn't wait for the timer to finish.
4. After 2 seconds, the callback inside `setTimeout` is executed.

If you have more questions about this concept or need further clarification, feel free to ask!